

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ
по лабораторной работе №2
по дисциплине «Построение и анализ алгоритмов»
Тема: Задача Коммивояжера

Студент гр. 4343

Едыгеев Р. Б.

Преподаватель

Калишенко Е. Л.

Санкт-Петербург

2026

Цель работы.

Необходимо реализовать программу, решающую 3К двумя методами в соответствии с вариантом: 1) Методом ВиГ. 2) Приближённым методом.

Работа выполнена в соответствии с вариантом 1.

Задание 1.

Решить задачу коммивояжера алгоритмом Литтла.

Входные данные:

Первая строка содержит одно целое число N (N — количество городов). Города нумеруются последовательными числами от 0 до $N-1$. Следующие N строк содержат по N чисел каждая, разделённых пробелами, формируя таким образом матрицу стоимостей M . Каждый элемент $M_{i,j}$ этой матрицы представляет собой затраты маны на перемещение из города i в город j .

Выходные данные:

Первая строка: Список из N целых чисел, разделённых пробелами, обозначающих оптимальный порядок городов в магическом маршруте Гамильтона. В начале идёт город 0, с которого начинается маршрут, затем последующие города до тех пор, пока все они не будут посещены. Вторая строка: Число, указывающее на суммарное количество израсходованной маны для завершения пути.

Задание 2.

Разработайте программу, которая решает задачу коммивояжера при помощи 2-приближенного алгоритма АДО МОД. В данной постановке задачи нужно вернуться в исходную вершину после прохождения всех остальных вершин. При обходе остовного дерева (MST) необходимо идти по минимальному допустимому ребру из текущего. Каждая вершина в графе обозначается неотрицательным числом, начиная с 0, каждое ребро имеет неотрицательный вес. В графе нет рёбер из вершины в саму себя, в матрице весов на месте таких отсутствующих рёбер стоит значение -1.

В первой строке указывается начальная вершина. Далее идёт матрица весов.

В качестве выходных данных необходимо представить длину пути, полученного при помощи алгоритма. Следующей строкой необходимо представить путь, в котором перечислены вершины, по которым необходимо пройти от начальной вершины.

Основные теоретические положения.

Задача коммивояжёра (также известная как задача о бродячем торговце) — это знаменитая задача комбинаторной оптимизации, которая состоит в поиске самого короткого возможного маршрута, проходящего через заданный список городов, с заходом в каждый город ровно один раз и возвращением в исходный город. Название происходит от французского слова «коммивояжёр», означающего бродячего торговца.

Формально задача может быть определена на графе, где вершины представляют города, а взвешенные рёбра — расстояния или стоимость путей между ними. Цель состоит в минимизации суммарного веса (расстояния, стоимости, времени и т. д.) маршрута. Это NP-трудная задача, что означает отсутствие известного алгоритма, который может решить все случаи задачи полиномиально быстро относительно количества городов, и считается, что такой алгоритм не может быть разработан.

Алгоритм Литтла — это точный метод решения задачи коммивояжёра, основанный на методе ветвей и границ. Он находит гамильтонов цикл минимальной стоимости в полном ориентированном графе. Алгоритм последовательно разбивает множество всех допустимых решений на подмножества (ветвление) и оценивает снизу стоимость решений в каждом подмножестве (вычисление границы), чтобы отсекал заведомо неоптимальные варианты. Одна из особенностей алгоритма — приведение матрицы путей: из каждой строки (каждого столбца) вычитается наименьший по стоимости путь этой строки (столбца), который добавляется в нижнюю

границу. После для каждого нулевого элемента рассчитывается коэффициент (штраф) как сумма минимальных элементов строки и столбца этого нулевого элемента (исключая его самого). Критерием выбора дуги для ветвления в данном алгоритме является нулевой элемент с наибольшим штрафом, так как его исключение сулит максимальный рост нижней границы. Это дуга-претендент на включение в оптимальный маршрут. Обозначив выбранный переход как (i, j) , формируем два новых узла: в первом фиксируем переезд (удаляя строку i и столбец j и запрещая обратный ход), во втором — запрещаем его. Приоритет при дальнейшем обходе дерева решений отдаётся узлу с минимальной текущей оценкой. Алгоритм завершается на узле с матрицей 2×2 : две оставшиеся дуги замыкают контур. Стоимость — сумма всех констант приведения по пройденному пути в дереве. Оптимальность гарантируется тем, что в этот момент не осталось нерассмотренных узлов с оценкой ниже найденной стоимости.

2-приближенный алгоритм не дает точного решения, а лишь некоторое приближенное к истине. Алгоритм двойного обхода применим в том случае, если матрица весов симметрична (т.е. $a_{ij} = a_{ji}$) и удовлетворяет неравенству треугольника (т.е. $a_{ij} \leq a_{ik} + a_{kj}$). С помощью стороннего алгоритма строится минимальное остовное дерево, производится двойной обход МОД (например с помощью обхода в глубину). В списке пройденных вершин вычеркиваются повторения (остаются только первые вхождения). Полученный путь является приближенным решением задачи коммивояжера с оценкой, равной стоимости маршрута АДО МОД, деленной на стоимость оптимального решения, которая не больше 2.

Выполнение работы.

Для решения задачи были разработаны программы на языке Python (см. Приложение А).

Пайплайн работы алгоритма.

Алгоритм Литтла. На первом этапе исходная матрица приводится по строкам и столбцам: для каждой строки и столбца находится минимум, который вычитается из соответствующих элементов, а сумма вычтенных минимумов формирует начальную нижнюю границу. Приведённая матрица и граница помещаются в корень дерева ветвлений. Создаётся очередь с приоритетом, где приоритет задаётся идентификатором узла, гарантирующим обработку в порядке создания, и в неё помещается корневой узел.

Далее запускается основной цикл: из очереди извлекается узел с наименьшим идентификатором. Если его нижняя граница не меньше текущего рекорда, узел отсекается. Если узел содержит $n-1$ дугу в пути, оставшаяся допустимая дуга добавляется, строится полный тур, вычисляется его фактическая стоимость по исходной матрице, и если она меньше рекорда, рекорд обновляется. Если путь неполный, для текущей приведённой матрицы вычисляются штрафы для всех нулевых клеток: для каждого нуля суммируются минимальные элементы его строки и столбца, исключая сам этот ноль. Выбирается дуга с максимальным штрафом. Создаётся правая ветвь: в матрице копии путь запрещается выставлением бесконечности, матрица приводится заново, к границе родителя добавляется новая константа приведения, и, если результат меньше рекорда, новый узел добавляется в очередь. Затем строится левая ветвь: дуга включается в путь, строка и столбец выбранной дуги заполняются бесконечностями. Определяется запрещаемая обратная дуга для предотвращения подциклов — для этого отслеживаются уже сформированные фрагменты пути, и если добавление дуги не замыкает полный цикл преждевременно, запрещается дуга от конца текущей цепочки к её началу, иначе просто запрещается обратный ход. Матрица приводится, вычисляется новая граница, и при допустимости узел добавляется в очередь. Цикл продолжается до опустошения очереди. После завершения из накопленных дуг рекордного пути выдается итоговый тур и его стоимость.

Алгоритм АДО МОД. На первом этапе вызывается алгоритм Прима для построения МОД. На каждом шаге извлекается вершина с наименьшим весом ребра, она помечается посещённой, а ребро, по которому в неё пришли, добавляется в дерево. Для всех непосещённых соседей обновляются минимальные веса рёбер, и они помещаются в очередь. Если по окончании обхода не все вершины оказались посещёнными, граф несвязен, и алгоритм завершается с ошибкой.

Затем выполняется обход в глубину удвоенного остовного дерева для построения эйлерова цикла. Начиная с той же стартовой вершины, рекурсивно обходятся все смежные вершины: при входе в вершину она добавляется в путь, затем рекурсивно посещаются все её соседи по дереву, и после возврата из рекурсии вершина снова добавляется в путь. Таким образом формируется эйлеров обход, в котором каждое ребро дерева проходится дважды — по одному разу в каждом направлении.

На последнем этапе вычеркиваются повторения вершин и вычисляется итоговая стоимость.

Анализ сложности алгоритма.

Алгоритм Литтла. Так как алгоритм является частным случаем МВиГ, то в худшем случае он перебирает все возможные перестановки ребер между вершинами, сложность – $O(n!)$. Благодаря отсечениям работа метода куда быстрее.

Сложность по памяти. Основной вес занимает хранение узлов состояний матриц – $O(n^2)$ для каждой. Количество узлов дерева ветвлений зависит от входных данных и качества отсечений, но в общем случае можно сказать, что с ростом n число узлов растёт экспоненциально. В итоге получается сложность $O(M \cdot n^2)$, где M – количество одновременно хранящихся узлов, n – размер матрицы.

Алгоритм АДО МОД. Построение МОД зависит от алгоритма, но т.к. в реализованной программе использовался Прима, то сложность равна $O(n^2)$. Построение эйлерова пути реализовано рекурсивным обходом в глубину и не займет более $O(n)$ сложности. Удаление повторов реализовано простым проходом по массиву – $O(n)$. Итоговая сложность составляет $O(n^2)$

Сложность по памяти. Хранится матрица смежности стоимостей путей – $O(n^2)$.

Тестирование.

Тестирование алгоритма Литтла.

1. Проверка логирования (Рис. 1 - 2):

```
src > out.txt
1  0  1  2  3
2  0  inf  10.0  15.0  20.0
3  1  10.0  inf  35.0  25.0
4  2  15.0  35.0  inf  30.0
5  3  20.0  25.0  30.0  inf
6
7
8  -----
9  ✓ (ident=27326 | PID=1) RIGHT BRANCH ---- excluded edge - 0 - 1, penalty = 5.0
10 0  inf  inf  0.0  0.0
11 1  0.0  inf  20.0  5.0
12 2  0.0  15.0  inf  5.0
13 3  0.0  0.0  5.0  inf
14 ✓ 3  0.0  0.0  5.0  inf
15
16  LOWER_BOUND = 75.0
17
18
19  -----
20
21  -----
22  ✓ (ident=37311 | PID=1) LEFT BRANCH ---- forbidden row - 0, col - 1
23  FORBIDDEN ROW 0 and col 1
24  FORBIDDEN EDGE: (1, 0)
25  CURRENT PATH = [(0, 1)]
26
27 0  inf  inf  inf  inf
28 1  inf  inf  10.0  0.0
29 2  0.0  inf  inf  5.0
30 3  0.0  inf  0.0  inf
31 ✓ 3  0.0  inf  0.0  inf
32
33  LOWER_BOUND = 80.0
```

Рисунок 1. Логирование

```
src > out.txt
188 v 3 inf inf inf inf
189 |
190 | | LOWER_BOUND = 80.0
191 |
192 |
193 |-----
194 |
195 |
196 v =====
197 | | NODE (ident=87805, PID=27326) CUTTED: LB=90.0 >= best_cost=80.0
198 |
199 |=====
200 |
201 |
202 |
203 v =====
204 | | NODE (ident=95960, PID=25106) CUTTED: LB=80.0 >= best_cost=80.0
205 |
206 |=====
207 |
208 |
209 |
210 v =====
211 | | NODE (ident=99028, PID=9192) CUTTED: LB=95.0 >= best_cost=80.0
212 |
213 |=====
214 |
215 |
216 |
217 |===== ANSWER =====
218 | 0 - 1 - 3 - 2 - 0
219 | 80.0
```

Рисунок 2. Логирование

2. Тестирование с отсутствием ребер (рис. 3). Входные данные:

```
4
-1 -1 -1 -1
-1 -1 35 25
15 35 -1 -1
20 25 30 -1
```

```
crakeny@LAPTOP-TQ9SBN6I:/mnt/c/U
test.txt
no path
```

Рисунок 3. Тест с отсутствием ребер

3. Тестовые данные с задания на Stepik №1 (Рис. 4 – 6):

Тестовые данные			
№ Теста	Входные данные	Выходные данные	
1	3 -1 1 3 3 -1 1 1 2 -1	0 1 2 3.0	

Рисунок 4. Тестовые данные

1	0	1	2
2	inf	1.0	3.0
3	1	3.0	inf
4	2	1.0	2.0
5			
6			

Рисунок 5. Входные данные

```

76
77
78  ===== ANSWER =====
79  0 - 1 - 2 - 0
80  3.0

```

Рисунок 6. Выходные данные

4. Тестовые данные с задания на Stepik №2 (Рис. 7 – 9):

2	4 -1 3 4 1 1 -1 3 4 9 2 -1 4 8 9 2 -1	0 3 2 1 6.0
---	---	----------------

Рисунок 7. Тестовые данные

```
src > out.txt
```

1	0	1	2	3
2	0	inf	3.0	4.0
3	1	1.0	inf	3.0
4	2	9.0	2.0	inf
5	3	8.0	9.0	2.0
6				
7				

Рисунок 8. Входные данные

```
334
335 ===== ANSWER =====
336 0 -> 3 -> 2 -> 1 -> 0
337 6.0
```

Рисунок 9. Выходные данные

Тестирование алгоритма АДО МОД.

1. Проверка логирования (рис. 10):

```
src > out.txt
1  START: 2
2  0  1  2  3
3  0  3.00  4.00  1.00
4  1  1.00  3.00  4.00
5  2  9.00  2.00  4.00
6  3  8.00  9.00  2.00
7
8
9  --- Построение МОД (алгоритм Прима) ---
10  Ребро (0, 1) вес: 1.00
11  Ребро (0, 3) вес: 1.00
12  Ребро (1, 2) вес: 2.00
13  Вес МОД: 4.00
14
15  --- Эйлеров обход ---
16  2 -> 1 -> 0 -> 3 -> 0 -> 1 -> 2
17
18  --- Shortcutting (удаление повторов) ---
19  2 -> 1 -> 0 -> 3
20
21  --- Замыкание цикла ---
22  2 -> 1 -> 0 -> 3 -> 2
23
24  --- Расчёт стоимости ---
25  (2, 1) = 2.00
26  (1, 0) = 1.00
27  (0, 3) = 1.00
28  (3, 2) = 2.00
29  Общая стоимость: 6.00
30
31
32  ===== ANSWER =====
33  2 -> 1 -> 0 -> 3 -> 2
34  Общая стоимость: 6.00
35
```

Рисунок 10. Логирование

2. Ввод стартовой точки, лежащей вне диапазона (Рис. 11 – 12):

```
src > test.txt
1 4
2 -1 3 4 1
3 1 -1 3 4
4 9 2 -1 4
5 8 9 2 -1
```

Рисунок 11. Входные данные

```
no path, vertex out of range
crakeny@LAPTOP-TQ9SBN6I:/mnt/c/Users/Hp/OneDrive/Рабочий стол/projects/2_COURSE/BAAA/1b2/src$
```

Рисунок 12. Выходные данные

3. Тесты с задания на Stepik №1 (Рис. 13 - 14):

№ Теста	Входные данные	Выходные данные
1	<pre>10 -1 9.22 23.32 8.49 16.76 30.0 26.02 13.89 3.61 13.15 21.4 8.06 4.12 9.22 -1 32.2 17.69 25.5 39.2 33.38 13.93 12.73 21.02 30.48 5.1 11.66 23.32 32.2 -1 15.23 15.52 12.17 25.24 27.29 19.72 12.21 3.16 29.0 20.62 8.49 17.69 15.23 -1 10.05 21.63 21.19 18.03 5.0 8.06 13.04 15.65 7.28 16.76 25.5 15.52 10.05 -1 15.26 11.66 28.07 14.14 16.12 12.37 24.74 17.03 30.0 39.2 12.17 21.63 15.26 -1 19.1 37.64 26.63 22.56 11.05 37.22 28.65 26.02 33.38 25.24 21.19 11.66 19.1 -1 38.83 24.33 27.78 22.2 34.0 27.46 13.89 13.93 27.29 18.03 28.07 37.64 38.83 -1 14.56 15.23 26.93 8.94 11.4 3.61 12.73 19.72 5.0 14.14 26.63 24.33 14.56 -1</pre>	<pre>147.25 10 2 5 9 3 8 12 0 11 1 7 4 6 10</pre>

Рисунок 13. Тестовые данные

```

--- Расчёт стоимости ---
· (10, 2) = 3.16
· (2, 5) = 12.17
· (5, 9) = 22.56
· (9, 3) = 8.06
· (3, 8) = 5.00
· (8, 12) = 3.16
· (12, 0) = 4.12
· (0, 11) = 8.06
· (11, 1) = 5.10
· (1, 7) = 13.93
· (7, 4) = 28.07
· (4, 6) = 11.66
· (6, 10) = 22.20
· Общая стоимость: 147.25

===== ANSWER =====
10 -> 2 -> 5 -> 9 -> 3 -> 8 -> 12 -> 0 -> 11 -> 1 -> 7 -> 4 -> 6 -> 10
Общая стоимость: 147.25

```

Рисунок 14. Выходные данные

4. Тесты с задания на Stepik №2 (Рис. 15 – 16):

Пример входных данных

```

2
-1 18.97 22.36 19.42 3.61
18.97 -1 35.61 38.01 17.0
22.36 35.61 -1 16.28 21.19
19.42 38.01 16.28 -1 21.02
3.61 17.0 21.19 21.02 -1

```

В первой строке указывается начальная вершина.

Далее идёт матрица весов.

В качестве выходных данных необходимо представить длину пути, которую необходимо представить путь, в котором перечислены вершины, по которым приведённых в примере входных данных ответом будет

```

91.92

2 3 0 4 1 2

```

Рисунок 15. Тестовые данные

```

26  ✓ --- Расчёт стоимости ---
27  | (2, 3) = 16.28
28  | (3, 0) = 19.42
29  | (0, 4) = 3.61
30  | (4, 1) = 17.00
31  | (1, 2) = 35.61
32  | Общая стоимость: 91.92
33
34
35  ===== ANSWER =====
36  2 -> 3 -> 0 -> 4 -> 1 -> 2
37  Общая стоимость: 91.92
38  |

```

Рисунок 16. Выходные данные

Вывод.

В ходе выполнения лабораторной работы были изучены способы решения задачи коммивояжера, такие как алгоритм Литтла и алгоритм двойного обхода минимального остовного дерева. Были разработаны программы на Python, выполнен анализ сложности алгоритма по времени и памяти, проведено тестирование программ, осуществлено логирование процесса выполнения программы.

Был получен опыт построения алгоритма для решения задачи, а также опыт в сфере оценки и анализа проведенной работы.

ПРИЛОЖЕНИЕ А КОД ПРОГРАММЫ

Файл little_alg.py:

```
import sys
import heapq
import random

file = "out.txt"

def show(matrix, n, f):
    f.write("\n")
    f.write("\t")
    f.writelines([f"{x}\t\t" for x in range(n)])
    f.write("\n")
    for i in range(n):
        f.write(f"{i}\t")
        f.writelines([f"{x}\t\t" for x in matrix[i]])
        f.write("\n")
    f.write("\n")

def show_lb(lower_bound, f):
    f.write(f"\t\tLOWER_BOUND = {lower_bound}\n\n")

def reduce_matrix_fast(matrix, n):
    lower_bound = 0
    for i in range(n):
        min_val = float('inf')
        row = matrix[i]
        for j in range(n):
            if row[j] < min_val:
                min_val = row[j]
        if min_val > 0 and min_val != float('inf'):
            lower_bound += min_val
            for j in range(n):
                if row[j] != float('inf'):
                    row[j] -= min_val
    for j in range(n):
        min_val = float('inf')
        for i in range(n):
            if matrix[i][j] < min_val:
                min_val = matrix[i][j]
        if min_val > 0 and min_val != float('inf'):
            lower_bound += min_val
            for i in range(n):
                if matrix[i][j] != float('inf'):
                    matrix[i][j] -= min_val
    return lower_bound

def find_best_arc(matrix, n):
    max_penalty = -1
    best_arc = None
    for i in range(n):
        row = matrix[i]
        for j in range(n):
```

```

        if row[j] == 0:
            min_row = float('inf')
            for k in range(n):
                if k != j:
                    val = row[k]
                    if val < min_row:
                        min_row = val
            min_col = float('inf')
            for k in range(n):
                if k != i:
                    val = matrix[k][j]
                    if val < min_col:
                        min_col = val
            penalty = min_row + min_col
            if penalty > max_penalty:
                max_penalty = penalty
            best_arc = (i, j, max_penalty)
    return best_arc

def get_forbidden_arc(path, new_arc, n):
    i, j, _ = new_arc
    incoming = [-1] * n
    outgoing = [-1] * n
    for u, v in path:
        outgoing[u] = v
        incoming[v] = u
    outgoing[i] = j
    incoming[j] = i
    curr = j
    start = i
    visited = set()
    while curr != -1 and curr not in visited:
        visited.add(curr)
        curr = outgoing[curr]
    if curr == -1:
        return (j, i)
    cycle_end = curr
    curr = start
    while curr != cycle_end and curr != -1:
        curr = incoming[curr]
    if curr == -1:
        return (j, cycle_end)
    return (j, i)

def solve_tsp(n, matrix_orig):
    ids = [x for x in range(2, 100000)]

    n = len(matrix_orig)
    matrix = [row[:] for row in matrix_orig]

    lower_bound = reduce_matrix_fast(matrix, n)

    pq = []
    ident = 1
    pid = 0
    heapq.heappush(pq, (ident, pid, lower_bound, 0, matrix, []))
    best_cost = float('inf')

```



```

best_path = None
while pq:
    ident, pid, lb, _, mat, path = heapq.heappop(pq)

    if lb >= best_cost:
        f = open(file, "a")

f.write("\n\n=====
=====
        f.write(f"\t\t\tNODE    (ident={ident},    PID={pid})    CUTTED:
LB={lb} >= best_cost={best_cost}\n")

f.write("\n=====
=====
        f.close()
        continue
    if len(path) == n - 1:
        remaining_i = -1
        remaining_j = -1
        for i in range(n):
            for j in range(n):
                if mat[i][j] != float('inf'):
                    remaining_i = i
                    remaining_j = j
                    break
            if remaining_i != -1:
                break
        if remaining_i != -1:
            full_path = path + [(remaining_i, remaining_j)]
            incoming = [-1] * n
            outgoing = [-1] * n
            for u, v in full_path:
                outgoing[u] = v
                incoming[v] = u
            tour = []
            start = 0
            curr = start
            for _ in range(n):
                tour.append(curr)
                curr = outgoing[curr]
            if len(set(tour)) == n:
                cost = 0
                for u, v in full_path:
                    cost += matrix_orig[u][v]
                if cost < best_cost:
                    best_cost = cost
                    best_path = full_path
            continue
    arc = find_best_arc(mat, n)
    if arc is None:
        continue
    i, j, max_pen = arc
    mat_right = [row[:] for row in mat]
    mat_right[i][j] = float('inf')

    lb_right = lb + reduce_matrix_fast(mat_right, n)

```



```

    for _ in range(n):
        tour.append(curr)
        curr = outgoing[curr]
    return best_cost, tour

def main():
    data = sys.stdin.read().strip().split()
    if not data:
        return

    n = int(data[0])

    f = open(file, "w")
    f.write("\t")
    f.writelines([f"{x}\t\t" for x in range(n)])
    f.write("\n")

    matrix = []
    idx = 1
    for i in range(n):
        row = []
        for j in range(n):
            val = float(data[idx])
            if val <= 0:
                row.append(float('inf'))
            else:
                row.append(val)
            idx += 1
        matrix.append(row)
        f.write(f"{i}\t")
        f.writelines([f"{x}\t\t" for x in row])
        f.write("\n")
    f.write("\n")
    f.close()

    result = solve_tsp(n, matrix)
    if result == "no path":
        print("no path")
        return
    cost, tour = result
    res = ' '.join(map(str, tour))
    print(res)
    print(f"{cost:.1f}")

    f = open(file, "a")
    f.write("\n\n===== ANSWER =====\n")
    f.writelines([f"{x} - " for x in tour])
    f.write(f" {tour[0]}")
    f.write(f"\n{cost:.1f}")
    f.close()

if __name__ == "__main__":
    main()

```

Файл mot_alg.py:

```

import sys
import heapq

file = "out.txt"

def prims(graph, n, start):
    mst = [[] for _ in range(n)]
    visited = [False] * n
    min_edge = [float('inf')] * n
    min_edge[start] = 0
    parent = [-1] * n

    pq = [(0, start)]

    while pq:
        weight, u = heapq.heappop(pq)

        if visited[u]:
            continue

        visited[u] = True

        if parent[u] != -1:
            v = parent[u]
            mst[u].append((v, weight))
            mst[v].append((u, weight))

        for v in range(n):
            if graph[u][v] != -1 and not visited[v] and graph[u][v] <
min_edge[v]:
                min_edge[v] = graph[u][v]
                parent[v] = u
                heapq.heappush(pq, (graph[u][v], v))

```

```

    if not all(visited):
        return None

    for i in range(n):
        mst[i].sort(key=lambda x: (x[1], x[0]))
    return mst

def dfs(mst, n, u):
    visited = [False] * n
    euler_path = []

    def dfs_recursive(v):
        visited[v] = True
        euler_path.append(v)
        for next_v, w in mst[v]:
            if not visited[next_v]:
                dfs_recursive(next_v)
                euler_path.append(v)

    dfs_recursive(u)
    return euler_path

def solve_tsp(graph, n, start):
    mst = prims(graph, n, start)

    if mst is None:
        f = open(file, "a")
        f.write("\nМОД не построен - граф несвязный\n")
        f.close()
        return None

    f = open(file, "a")
    f.write("\n--- Построение МОД (алгоритм Прима) ---\n")

```

```

mst_weight = 0
for u in range(n):
    for v, w in mst[u]:
        if u < v:
            f.write(f"  Ребро ({u}, {v}) вес: {w:.2f}\n")
            mst_weight += w
f.write(f"  Вес МОД: {mst_weight:.2f}\n")
f.close()

```

```

euler_path = dfs(mst, n, start)

```

```

f = open(file, "a")
f.write(f"\n--- Эйлера обход ---\n")
f.write(f"  {' -> '.join(map(str, euler_path))}\n")
f.close()

```

```

path = []
seen = set()
for v in euler_path:
    if v not in seen:
        path.append(v)
        seen.add(v)

```

```

f = open(file, "a")
f.write(f"\n--- Shortcutting (удаление повторов) ---\n")
f.write(f"  {' -> '.join(map(str, path))}\n")
f.close()

```

```

path.append(start)

```

```

f = open(file, "a")

```

```

f.write(f"\n--- Замыкание цикла ---\n")
f.write(f"  {' -> '}.join(map(str, path))}\n")
f.close()

total_length = 0.0
f = open(file, "a")
f.write(f"\n--- Расчёт стоимости ---\n")

for i in range(len(path) - 1):
    u, v = path[i], path[i + 1]
    if graph[u][v] == -1 or graph[u][v] == float('inf'):
        f.write(f"  ОШИБКА: нет прямого ребра ({u}, {v})\n")
        f.close()
        return None
    edge_weight = graph[u][v]
    total_length += edge_weight
    f.write(f"  ({u}, {v}) = {edge_weight:.2f}\n")

f.write(f"  Общая стоимость: {total_length:.2f}\n")
f.close()

print(f"{total_length:.2f}")
print(' '.join(map(str, path)))

return total_length, path

def main():
    data = sys.stdin.read().strip().split()
    if not data:
        return

    start_vertex = int(data[0])

```

```

n = int((len(data) - 1) ** 0.5)
if start_vertex >= n:
    print("no path, vertex out of range")
    return

f = open(file, "w")
f.write(f"START: {start_vertex}\n")
f.write("\t")
f.writelines([f"{x}\t\t" for x in range(n)])
f.write("\n")

matrix = []
idx = 1
for i in range(n):
    row = []
    for j in range(n):
        val = float(data[idx])
        if val <= 0:
            row.append(-1)
        else:
            row.append(val)
        idx += 1
    matrix.append(row)
    f.write(f"{i}\t")
    f.writelines([f"{x:.2f}\t\t" if x != -1 else "-\t\t" for x in
row])
    f.write("\n")
f.write("\n")
f.close()

result = solve_tsp(matrix, n, start_vertex)
if result is None:
    f = open(file, "a")
    f.write("\nno path\n")

```



```

        f.close()

        print("no path")

        return

cost, tour = result

f = open(file, "a")
f.write("\n\n===== ANSWER =====\n")
f.write(" -> ".join(map(str, tour)))
f.write(f"\nОбщая стоимость: {cost:.2f}\n")
f.close()

if __name__ == "__main__":
    main()

```